Advanced Data Structures

Binary Search Trees, Python Generators, and Hash Tables

Martin Benning

University College London
ENGF0034 – Design and Professional Skills

27 October 2026



Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast linear and hierarchical search trade-offs, and state the formal BST invariant.
2. Implement recursive BST lookup, insertion, and three deletion cases (including Hibbard value swapping).
3. Explain lazy evaluation in Python generators (`yield`, `yield from`) and analyse auxiliary memory.
4. Derive BST height bounds, identify degenerate worst cases, and motivate self-balancing trees.
5. Design hash tables with separate chaining, analyse expected $\mathcal{O}(1)$ lookup under uniform hashing, and explain CPython dict internals.

Lecture Roadmap

The Search Problem & Tree Foundations
Binary Search Trees & Invariants
Tree Traversals & Python Generators
Advanced BST: Deletion & Hibbard Swapping
Complexity & The Balance Trap
Hash Tables and Collision Resolution
Synthesis, Comparison & Outlook

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast linear and hierarchical search trade-offs, and state the formal BST invariant.
2. Implement recursive BST lookup, insertion, and three deletion cases (including Hibbard value swapping).
3. Explain lazy evaluation in Python generators (`yield`, `yield from`) and analyse auxiliary memory.
4. Derive BST height bounds, identify degenerate worst cases, and motivate self-balancing trees.
5. Design hash tables with separate chaining, analyse expected $\mathcal{O}(1)$ lookup under uniform hashing, and explain CPython dict internals.

Lecture Roadmap

The Search Problem & Tree Foundations
Binary Search Trees & Invariants
Tree Traversals & Python Generators
Advanced BST: Deletion & Hibbard Swapping
Complexity & The Balance Trap
Hash Tables and Collision Resolution
Synthesis, Comparison & Outlook

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast linear and hierarchical search trade-offs, and state the formal BST invariant.
2. Implement recursive BST lookup, insertion, and three deletion cases (including Hibbard value swapping).
3. Explain lazy evaluation in Python generators (`yield`, `yield from`) and analyse auxiliary memory.
4. Derive BST height bounds, identify degenerate worst cases, and motivate self-balancing trees.
5. Design hash tables with separate chaining, analyse expected $\mathcal{O}(1)$ lookup under uniform hashing, and explain CPython dict internals.

Lecture Roadmap

The Search Problem & Tree Foundations
Binary Search Trees & Invariants
Tree Traversals & Python Generators
Advanced BST: Deletion & Hibbard Swapping
Complexity & The Balance Trap
Hash Tables and Collision Resolution
Synthesis, Comparison & Outlook

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast linear and hierarchical search trade-offs, and state the formal BST invariant.
2. Implement recursive BST lookup, insertion, and three deletion cases (including Hibbard value swapping).
3. Explain lazy evaluation in Python generators (`yield`, `yield from`) and analyse auxiliary memory.
4. Derive BST height bounds, identify degenerate worst cases, and motivate self-balancing trees.
5. Design hash tables with separate chaining, analyse expected $\mathcal{O}(1)$ lookup under uniform hashing, and explain CPython dict internals.

Lecture Roadmap

The Search Problem & Tree Foundations
Binary Search Trees & Invariants
Tree Traversals & Python Generators
Advanced BST: Deletion & Hibbard Swapping
Complexity & The Balance Trap
Hash Tables and Collision Resolution
Synthesis, Comparison & Outlook

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast linear and hierarchical search trade-offs, and state the formal BST invariant.
2. Implement recursive BST lookup, insertion, and three deletion cases (including Hibbard value swapping).
3. Explain lazy evaluation in Python generators (`yield`, `yield from`) and analyse auxiliary memory.
4. Derive BST height bounds, identify degenerate worst cases, and motivate self-balancing trees.
5. Design hash tables with separate chaining, analyse expected $\mathcal{O}(1)$ lookup under uniform hashing, and explain CPython dict internals.

Lecture Roadmap

The Search Problem & Tree Foundations
Binary Search Trees & Invariants
Tree Traversals & Python Generators
Advanced BST: Deletion & Hibbard Swapping
Complexity & The Balance Trap
Hash Tables and Collision Resolution
Synthesis, Comparison & Outlook

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast linear and hierarchical search trade-offs, and state the formal BST invariant.
2. Implement recursive BST lookup, insertion, and three deletion cases (including Hibbard value swapping).
3. Explain lazy evaluation in Python generators (`yield`, `yield from`) and analyse auxiliary memory.
4. Derive BST height bounds, identify degenerate worst cases, and motivate self-balancing trees.
5. Design hash tables with separate chaining, analyse expected $\mathcal{O}(1)$ lookup under uniform hashing, and explain CPython dict internals.

Lecture Roadmap

The Search Problem & Tree Foundations
Binary Search Trees & Invariants
Tree Traversals & Python Generators
Advanced BST: Deletion & Hibbard Swapping
Complexity & The Balance Trap
Hash Tables and Collision Resolution
Synthesis, Comparison & Outlook

The Search Problem & Tree Foundations

The Search Problem: Contiguous vs Linked Trade-offs

Revisiting Weeks 2 and 3

Linear structures force a performance compromise:

- **Unsorted Array:** $\mathcal{O}(1)^{A}$ (amortised) append, but $\mathcal{O}(N)$ linear scan.
- **Sorted Array:** $\mathcal{O}(\log N)$ binary search, but $\mathcal{O}(N)$ shifts on insert and delete.
- **Linked List:** $\mathcal{O}(1)$ insert at a known pointer, but $\mathcal{O}(N)$ search without pointer.

The Core Dilemma

Can we achieve $\mathcal{O}(\log N)$ search without paying $\mathcal{O}(N)$ shifts on insertion?

Comparison of Linear Structures

Structure	Search	Insert	Delete
Unsorted Array	$\mathcal{O}(N)$	$\mathcal{O}(1)^{A}$	$\mathcal{O}(N)$
Sorted Array	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Linked List	$\mathcal{O}(N)$	$\mathcal{O}(1)$	$\mathcal{O}(N)$
Target (BST)	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$

^{A} Amortised time; linked list $\mathcal{O}(1)$ insert assumes a given node pointer.

The Solution Concept

Organise nodes into a **tree hierarchy** that halves the remaining search space at every step.

The Search Problem: Contiguous vs Linked Trade-offs

Revisiting Weeks 2 and 3

Linear structures force a performance compromise:

- **Unsorted Array:** $\mathcal{O}(1)^{A}$ (amortised) append, but $\mathcal{O}(N)$ linear scan.
- **Sorted Array:** $\mathcal{O}(\log N)$ binary search, but $\mathcal{O}(N)$ shifts on insert and delete.
- **Linked List:** $\mathcal{O}(1)$ insert at a known pointer, but $\mathcal{O}(N)$ search without pointer.

The Core Dilemma

Can we achieve $\mathcal{O}(\log N)$ search without paying $\mathcal{O}(N)$ shifts on insertion?

Comparison of Linear Structures

Structure	Search	Insert	Delete
Unsorted Array	$\mathcal{O}(N)$	$\mathcal{O}(1)^{A}$	$\mathcal{O}(N)$
Sorted Array	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Linked List	$\mathcal{O}(N)$	$\mathcal{O}(1)$	$\mathcal{O}(N)$
Target (BST)	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$

^{A} Amortised time; linked list $\mathcal{O}(1)$ insert assumes a given node pointer.

The Solution Concept

Organise nodes into a **tree hierarchy** that halves the remaining search space at every step.

The Search Problem: Contiguous vs Linked Trade-offs

Revisiting Weeks 2 and 3

Linear structures force a performance compromise:

- **Unsorted Array:** $\mathcal{O}(1)^{\mathcal{A}}$ (amortised) append, but $\mathcal{O}(N)$ linear scan.
- **Sorted Array:** $\mathcal{O}(\log N)$ binary search, but $\mathcal{O}(N)$ shifts on insert and delete.
- **Linked List:** $\mathcal{O}(1)$ insert at a known pointer, but $\mathcal{O}(N)$ search without pointer.

The Core Dilemma

Can we achieve $\mathcal{O}(\log N)$ search without paying $\mathcal{O}(N)$ shifts on insertion?

Comparison of Linear Structures

Structure	Search	Insert	Delete
Unsorted Array	$\mathcal{O}(N)$	$\mathcal{O}(1)^{\mathcal{A}}$	$\mathcal{O}(N)$
Sorted Array	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Linked List	$\mathcal{O}(N)$	$\mathcal{O}(1)$	$\mathcal{O}(N)$
Target (BST)	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$

$^{\mathcal{A}}$ Amortised time; linked list $\mathcal{O}(1)$ insert assumes a given node pointer.

The Solution Concept

Organise nodes into a **tree hierarchy** that halves the remaining search space at every step.

The Search Problem: Contiguous vs Linked Trade-offs

Revisiting Weeks 2 and 3

Linear structures force a performance compromise:

- **Unsorted Array:** $\mathcal{O}(1)^{\mathrm{A}}$ (amortised) append, but $\mathcal{O}(N)$ linear scan.
- **Sorted Array:** $\mathcal{O}(\log N)$ binary search, but $\mathcal{O}(N)$ shifts on insert and delete.
- **Linked List:** $\mathcal{O}(1)$ insert at a known pointer, but $\mathcal{O}(N)$ search without pointer.

The Core Dilemma

Can we achieve $\mathcal{O}(\log N)$ search without paying $\mathcal{O}(N)$ shifts on insertion?

Comparison of Linear Structures

Structure	Search	Insert	Delete
Unsorted Array	$\mathcal{O}(N)$	$\mathcal{O}(1)^{\mathrm{A}}$	$\mathcal{O}(N)$
Sorted Array	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Linked List	$\mathcal{O}(N)$	$\mathcal{O}(1)$	$\mathcal{O}(N)$
Target (BST)	$\mathcal{O}(\log \mathbf{N})$	$\mathcal{O}(\log \mathbf{N})$	$\mathcal{O}(\log \mathbf{N})$

^{A} Amortised time; linked list $\mathcal{O}(1)$ insert assumes a given node pointer.

The Solution Concept

Organise nodes into a **tree hierarchy** that halves the remaining search space at every step.

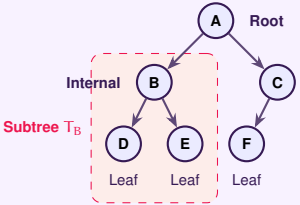
Trees: Definitions and Terminology

Mathematical Definition of a Tree

A **rooted tree** $T = (V, E)$ is a directed acyclic graph with a distinguished **root** vertex $r \in V$ of in-degree 0, where every other vertex $v \in V \setminus \{r\}$ has in-degree 1 and is reachable from r:

- **Root:** The unique vertex with in-degree 0.
- **Parent & Child:** A directed edge (u, v) indicates that u is the parent of v, and v is a child of u.
- **Leaf:** A node with out-degree 0 (no children).
- **Internal Node:** A node with out-degree $\geqslant 1$.
- **Subtree:** The tree induced by node u and all descendants reachable from u.

Tree Anatomy



Common Applications

File directories, HTML DOM trees, and Python abstract syntax trees (ASTs) are all rooted trees.

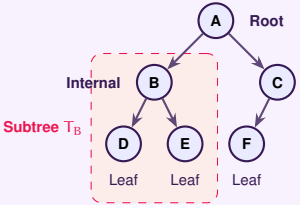
Trees: Definitions and Terminology

Mathematical Definition of a Tree

A **rooted tree** $T = (V, E)$ is a directed acyclic graph with a distinguished **root** vertex $r \in V$ of in-degree 0, where every other vertex $v \in V \setminus \{r\}$ has in-degree 1 and is reachable from r:

- **Root:** The unique vertex with in-degree 0.
- **Parent & Child:** A directed edge (u, v) indicates that u is the parent of v, and v is a child of u.
- **Leaf:** A node with out-degree 0 (no children).
- **Internal Node:** A node with out-degree $\geqslant 1$.
- **Subtree:** The tree induced by node u and all descendants reachable from u.

Tree Anatomy



Common Applications

File directories, HTML DOM trees, and Python abstract syntax trees (ASTs) are all rooted trees.

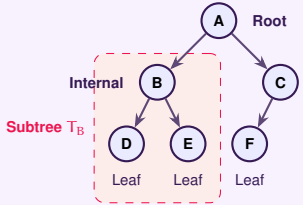
Trees: Definitions and Terminology

Mathematical Definition of a Tree

A **rooted tree** $T = (V, E)$ is a directed acyclic graph with a distinguished **root** vertex $r \in V$ of in-degree 0, where every other vertex $v \in V \setminus \{r\}$ has in-degree 1 and is reachable from r:

- **Root:** The unique vertex with in-degree 0.
- **Parent & Child:** A directed edge (u, v) indicates that u is the parent of v, and v is a child of u.
- **Leaf:** A node with out-degree 0 (no children).
- **Internal Node:** A node with out-degree $\geqslant 1$.
- **Subtree:** The tree induced by node u and all descendants reachable from u.

Tree Anatomy



Common Applications

File directories, HTML DOM trees, and Python abstract syntax trees (ASTs) are all rooted trees.

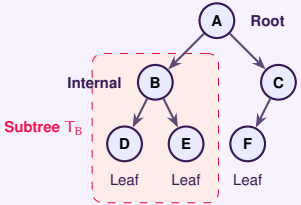
Trees: Definitions and Terminology

Mathematical Definition of a Tree

A **rooted tree** $T = (V, E)$ is a directed acyclic graph with a distinguished **root** vertex $r \in V$ of in-degree 0, where every other vertex $v \in V \setminus \{r\}$ has in-degree 1 and is reachable from r:

- **Root:** The unique vertex with in-degree 0.
- **Parent & Child:** A directed edge (u, v) indicates that u is the parent of v, and v is a child of u.
- **Leaf:** A node with out-degree 0 (no children).
- **Internal Node:** A node with out-degree $\geqslant 1$.
- **Subtree:** The tree induced by node u and all descendants reachable from u.

Tree Anatomy



Common Applications

File directories, HTML DOM trees, and Python abstract syntax trees (ASTs) are all rooted trees.

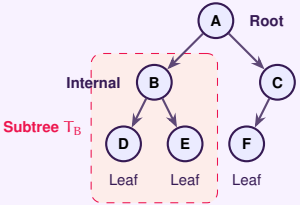
Trees: Definitions and Terminology

Mathematical Definition of a Tree

A **rooted tree** $T = (V, E)$ is a directed acyclic graph with a distinguished **root** vertex $r \in V$ of in-degree 0, where every other vertex $v \in V \setminus \{r\}$ has in-degree 1 and is reachable from r:

- **Root:** The unique vertex with in-degree 0.
- **Parent & Child:** A directed edge (u, v) indicates that u is the parent of v, and v is a child of u.
- **Leaf:** A node with out-degree 0 (no children).
- **Internal Node:** A node with out-degree $\geqslant 1$.
- **Subtree:** The tree induced by node u and all descendants reachable from u.

Tree Anatomy



Common Applications

File directories, HTML DOM trees, and Python abstract syntax trees (ASTs) are all rooted trees.

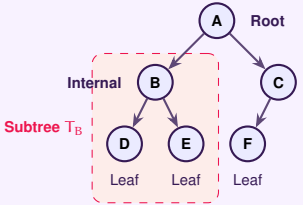
Trees: Definitions and Terminology

Mathematical Definition of a Tree

A **rooted tree** $T = (V, E)$ is a directed acyclic graph with a distinguished **root** vertex $r \in V$ of in-degree 0, where every other vertex $v \in V \setminus \{r\}$ has in-degree 1 and is reachable from r:

- **Root:** The unique vertex with in-degree 0.
- **Parent & Child:** A directed edge (u, v) indicates that u is the parent of v, and v is a child of u.
- **Leaf:** A node with out-degree 0 (no children).
- **Internal Node:** A node with out-degree $\geqslant 1$.
- **Subtree:** The tree induced by node u and all descendants reachable from u.

Tree Anatomy



Common Applications

File directories, HTML DOM trees, and Python abstract syntax trees (ASTs) are all rooted trees.

Tree Metrics: Depth, Height, and Binary Trees

Formal Definitions (Edge-Count)

- **Binary Tree:** At most two distinguished children per node (`left` and `right`).
- **Depth of Node u:** Number of edges on the path from root to u. $\text{depth}(\text{root}) = 0$.
- **Height of Tree** H**:** Maximum depth in T: $H = \max_{u \in T} \text{depth}(u)$ ($H = 0$ for a single node; $H = -1$ for empty).

Height Bounds for $N \geqslant 1$ Nodes

Level d holds at most 2^d nodes, so $N \leqslant 2^{H+1} - 1$:

$$\lfloor \log_2 N \rfloor \leqslant H \leqslant N - 1$$

Height Extremes for $N = 7$ Nodes

Algorithmic Consequence

Tree height H governs search comparisons. Goal: maintain $H = \mathcal{O}(\log N)$.

Tree Metrics: Depth, Height, and Binary Trees

Formal Definitions (Edge-Count)

- **Binary Tree:** At most two distinguished children per node (`left` and `right`).
- **Depth of Node u:** Number of edges on the path from root to u. $\text{depth}(\text{root}) = 0$.
- **Height of Tree** H**:** Maximum depth in T: $H = \max_{u \in T} \text{depth}(u)$ ($H = 0$ for a single node; $H = -1$ for empty).

Height Bounds for $N \geqslant 1$ Nodes

Level d holds at most 2^d nodes, so $N \leqslant 2^{H+1} - 1$:

$$\lfloor \log_2 N \rfloor \leqslant H \leqslant N - 1$$

Height Extremes for $N = 7$ Nodes

Algorithmic Consequence

Tree height H governs search comparisons. Goal: maintain $H = \mathcal{O}(\log N)$.

Tree Metrics: Depth, Height, and Binary Trees

Formal Definitions (Edge-Count)

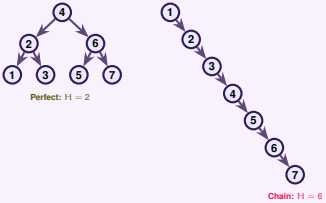
- **Binary Tree:** At most two distinguished children per node (`left` and `right`).
- **Depth of Node u:** Number of edges on the path from root to u. $\text{depth}(\text{root}) = 0$.
- **Height of Tree** H**:** Maximum depth in T: $H = \max_{u \in T} \text{depth}(u)$ ($H = 0$ for a single node; $H = -1$ for empty).

Height Bounds for $N \geqslant 1$ Nodes

Level d holds at most 2^d nodes, so $N \leqslant 2^{H+1} - 1$:

$$\lfloor \log_2 N \rfloor \leqslant H \leqslant N - 1$$

Height Extremes for $N = 7$ Nodes



Algorithmic Consequence

Tree height H governs search comparisons. Goal: maintain $H = \mathcal{O}(\log N)$.

Tree Metrics: Depth, Height, and Binary Trees

Formal Definitions (Edge-Count)

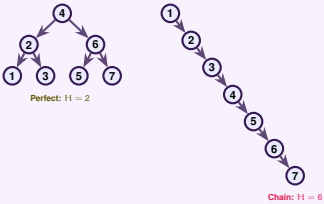
- **Binary Tree:** At most two distinguished children per node (`left` and `right`).
- **Depth of Node u:** Number of edges on the path from root to u. $\text{depth}(\text{root}) = 0$.
- **Height of Tree** H**:** Maximum depth in T: $H = \max_{u \in T} \text{depth}(u)$ ($H = 0$ for a single node; $H = -1$ for empty).

Height Bounds for $N \geqslant 1$ Nodes

Level d holds at most 2^d nodes, so $N \leqslant 2^{H+1} - 1$:

$$\lfloor \log_2 N \rfloor \leqslant H \leqslant N - 1$$

Height Extremes for $N = 7$ Nodes



Algorithmic Consequence

Tree height H governs search comparisons. Goal: maintain $H = \mathcal{O}(\log N)$.

Binary Search Trees & Invariants

The Binary Search Tree (BST) Invariant

Formal BST Ordering Invariant

Assuming distinct keys drawn from a totally ordered set, a binary tree is a **Binary Search Tree** if and only if for **every** node X in the tree:

1. For all $u \in \text{LeftSubtree}(X)$: $u.\text{key} < X.\text{key}$
2. For all $v \in \text{RightSubtree}(X)$: $v.\text{key} > X.\text{key}$

Global Property

The condition applies to **all ancestors and descendants**, not merely immediate children.

Subtree Key Ranges

Consequence for Search

Comparing a search key against $X.\text{key}$ eliminates an entire subtree in $\mathcal{O}(1)$ time.

The Binary Search Tree (BST) Invariant

Formal BST Ordering Invariant

Assuming distinct keys drawn from a totally ordered set, a binary tree is a **Binary Search Tree** if and only if for **every** node X in the tree:

1. For all $u \in \text{LeftSubtree}(X)$: $u.\text{key} < X.\text{key}$
2. For all $v \in \text{RightSubtree}(X)$: $v.\text{key} > X.\text{key}$

Global Property

The condition applies to **all ancestors and descendants**, not merely immediate children.

Subtree Key Ranges

Consequence for Search

Comparing a search key against $X.\text{key}$ eliminates an entire subtree in $\mathcal{O}(1)$ time.

The Binary Search Tree (BST) Invariant

Formal BST Ordering Invariant

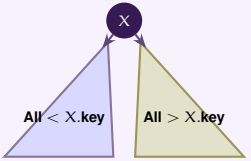
Assuming distinct keys drawn from a totally ordered set, a binary tree is a **Binary Search Tree** if and only if for **every** node X in the tree:

1. For all $u \in \text{LeftSubtree}(X)$: $u.\text{key} < X.\text{key}$
2. For all $v \in \text{RightSubtree}(X)$: $v.\text{key} > X.\text{key}$

Global Property

The condition applies to **all ancestors and descendants**, not merely immediate children.

Subtree Key Ranges



Consequence for Search

Comparing a search key against $X.\text{key}$ eliminates an entire subtree in $\mathcal{O}(1)$ time.

The Binary Search Tree (BST) Invariant

Formal BST Ordering Invariant

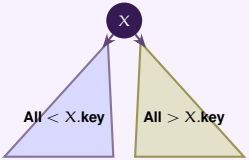
Assuming distinct keys drawn from a totally ordered set, a binary tree is a **Binary Search Tree** if and only if for **every** node X in the tree:

1. For all $u \in \text{LeftSubtree}(X)$: $u.\text{key} < X.\text{key}$
2. For all $v \in \text{RightSubtree}(X)$: $v.\text{key} > X.\text{key}$

Global Property

The condition applies to **all ancestors and descendants**, not merely immediate children.

Subtree Key Ranges

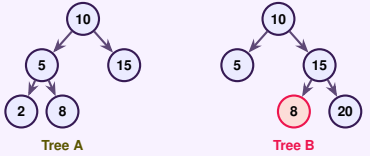


Consequence for Search

Comparing a search key against X.key eliminates an entire subtree in $\mathcal{O}(1)$ time.

Checkpoint 1: Spot the Invariant Violation

Inspect These Two Candidate Trees



The Diagnostic Question

In Tree B, node 15 satisfies $8 < 15 < 20$ locally. Is Tree B a valid BST? Why would `find(8)` fail?

Analysis

Tree B is invalid:

- Node 8 sits in the **right subtree of 10**.
- The invariant requires every node in 10's right subtree to be > 10. Since $8 < 10$, this violates the global invariant.

Search Failure Mode

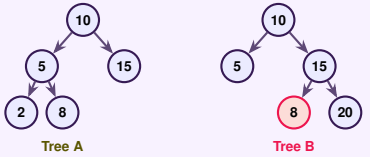
When searching for 8:

1. At root 10: $8 < 10 \implies$ search branches **left**.
2. Node 8 sits on the right, so search never finds it.

Valid key range for 15's left child is $(10, 15)$.

Checkpoint 1: Spot the Invariant Violation

Inspect These Two Candidate Trees



The Diagnostic Question

In Tree B, node 15 satisfies $8 < 15 < 20$ locally. Is Tree B a valid BST? Why would `find(8)` fail?

Analysis

Tree B is invalid:

- Node 8 sits in the **right subtree of 10**.
- The invariant requires every node in 10's right subtree to be > 10. Since $8 < 10$, this violates the global invariant.

Search Failure Mode

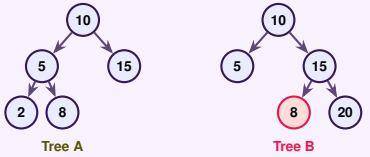
When searching for 8:

1. At root 10: $8 < 10 \implies$ search branches **left**.
2. Node 8 sits on the right, so search never finds it.

Valid key range for 15's left child is $(10, 15)$.

Checkpoint 1: Spot the Invariant Violation

Inspect These Two Candidate Trees



The Diagnostic Question

In Tree B, node 15 satisfies $8 < 15 < 20$ locally. Is Tree B a valid BST? Why would `find(8)` fail?

Analysis

Tree B is invalid:

- Node 8 sits in the **right subtree of 10**.
- The invariant requires every node in 10's right subtree to be > 10. Since $8 < 10$, this violates the global invariant.

Search Failure Mode

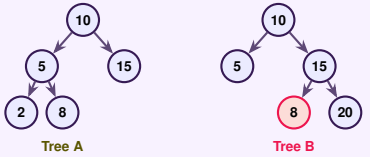
When searching for 8:

1. At root 10: $8 < 10 \implies$ search branches **left**.
2. Node 8 sits on the right, so search never finds it.

Valid key range for 15's left child is $(10, 15)$.

Checkpoint 1: Spot the Invariant Violation

Inspect These Two Candidate Trees



The Diagnostic Question

In Tree B, node 15 satisfies $8 < 15 < 20$ locally. Is Tree B a valid BST? Why would `find(8)` fail?

Analysis

Tree B is invalid:

- Node 8 sits in the **right subtree of 10**.
- The invariant requires every node in 10's right subtree to be > 10. Since $8 < 10$, this violates the global invariant.

Search Failure Mode

When searching for 8:

1. At root 10: $8 < 10 \implies$ search branches **left**.
2. Node 8 sits on the right, so search never finds it.

Valid key range for 15's left child is $(10, 15)$.

BST Representation: Tree and Node Classes

```python
class TreeNode:
  def __init__(self, key, value):
    self.key = key
    self.value = value
    self.left = None
    self.right = None

class BinaryTree:
  def __init__(self):
    self.root = None

  def lookup(self, key):
    if self.root is None:
      return None
    node = self.root.find(key)
    return node.value if node else None
```

Tree and Node Separation

Why use both `BinaryTree` and `TreeNode`?

- **Empty Trees:** When the tree contains no elements, `self.root = None` represents size 0 directly.
- **Root Reassignment:** When deleting the root node, `self.root` is reassigned directly.

Encapsulation

Callers invoke `tree.lookup(key)` on the wrapper without managing recursive node references directly.

BST Representation: Tree and Node Classes

```python
class TreeNode:
  def __init__(self, key, value):
    self.key = key
    self.value = value
    self.left = None
    self.right = None

class BinaryTree:
  def __init__(self):
    self.root = None

  def lookup(self, key):
    if self.root is None:
      return None
    node = self.root.find(key)
    return node.value if node else None
```

Tree and Node Separation

Why use both `BinaryTree` and `TreeNode`?

- **Empty Trees:** When the tree contains no elements, `self.root = None` represents size 0 directly.
- **Root Reassignment:** When deleting the root node, `self.root` is reassigned directly.

Encapsulation

Callers invoke `tree.lookup(key)` on the wrapper without managing recursive node references directly.

BST Lookup (`find`): Halving the Search Space

```
1  class TreeNode:
2    def find(self, key):
3      if key == self.key:
4        return self
5      if key < self.key:
6        if self.left is None:
7          return None
8        return self.left.find(key)
9      else:
10       if self.right is None:
11         return None
12       return self.right.find(key)
```

Trace: `find(12)`

Step-by-Step

1. At 10: $12 > 10 \implies$ branch right.
2. At 15: $12 < 15 \implies$ branch left.
3. At 12: $12 == 12 \implies$ match returned.

3 comparisons instead of 5 for a linear list.

Complexity Derivation

Each step descends one level. At most $H + 1$ nodes are visited:

$$T(N) = \mathcal{O}(H)$$

Balanced tree: $\mathcal{O}(\log N)$. Degenerate chain: $\mathcal{O}(N)$.

BST Lookup (`find`): Halving the Search Space

```
1  class TreeNode:
2    def find(self, key):
3      if key == self.key:
4        return self
5      if key < self.key:
6        if self.left is None:
7          return None
8        return self.left.find(key)
9      else:
10       if self.right is None:
11         return None
12       return self.right.find(key)
```

Complexity Derivation

Each step descends one level. At most $H + 1$ nodes are visited:

$$T(N) = \mathcal{O}(H)$$

Balanced tree: $\mathcal{O}(\log N)$. Degenerate chain: $\mathcal{O}(N)$.

Trace: `find(12)`

Step-by-Step

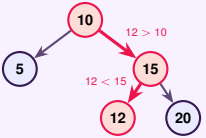
1. At 10: $12 > 10 \implies$ branch right.
2. At 15: $12 < 15 \implies$ branch left.
3. At 12: 12 == 12 $\implies$ match returned.

3 comparisons instead of 5 for a linear list.

BST Lookup (`find`): Halving the Search Space

```python
class TreeNode:
  def find(self, key):
    if key == self.key:
      return self
    if key < self.key:
      if self.left is None:
        return None
      return self.left.find(key)
    else:
      if self.right is None:
        return None
      return self.right.find(key)
```

Trace: `find(12)`



Step-by-Step

1. At 10: $12 > 10 \implies$ branch right.
2. At 15: $12 < 15 \implies$ branch left.
3. At 12: 12 == 12 $\implies$ match returned.

3 comparisons instead of 5 for a linear list.

Complexity Derivation

Each step descends one level. At most $H + 1$ nodes are visited:

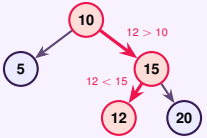
$$T(N) = \mathcal{O}(H)$$

Balanced tree: $\mathcal{O}(\log N)$. Degenerate chain: $\mathcal{O}(N)$.

BST Lookup (`find`): Halving the Search Space

```
class TreeNode:
  def find(self, key):
    if key == self.key:
      return self
    if key < self.key:
      if self.left is None:
        return None
      return self.left.find(key)
    else:
      if self.right is None:
        return None
      return self.right.find(key)
```

Trace: `find(12)`



Step-by-Step

1. At 10: $12 > 10 \implies$ branch right.
2. At 15: $12 < 15 \implies$ branch left.
3. At 12: `12 == 12` $\implies$ match returned.

3 comparisons instead of 5 for a linear list.

Complexity Derivation

Each step descends one level. At most $H + 1$ nodes are visited:

$$T(N) = \mathcal{O}(H)$$

Balanced tree: $\mathcal{O}(\log N)$. Degenerate chain: $\mathcal{O}(N)$.

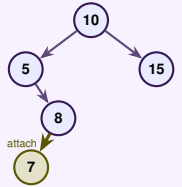
BST Insertion (`insert`): Growing at the Leaves

```python
class TreeNode:
  def insert(self, key, value):
    if key < self.key:
      if self.left is None:
        self.left = TreeNode(key, value)
      else:
        self.left.insert(key, value)
    else:
      if self.right is None:
        self.right = TreeNode(key, value)
      else:
        self.right.insert(key, value)
```

Insertion Mechanics

`insert` follows the path of a failed `find`. At `None`, it attaches a new leaf in $\mathcal{O}(1)$ pointer updates. Total time: $\mathcal{O}(H)$.

Trace: `insert(7, "G")`



Key Observation

Unlike dynamic arrays, inserting into a BST does not require shifting existing nodes in memory.

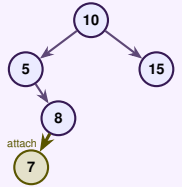
BST Insertion (`insert`): Growing at the Leaves

```python
class TreeNode:
  def insert(self, key, value):
    if key < self.key:
      if self.left is None:
        self.left = TreeNode(key, value)
      else:
        self.left.insert(key, value)
    else:
      if self.right is None:
        self.right = TreeNode(key, value)
      else:
        self.right.insert(key, value)
```

Insertion Mechanics

`insert` follows the path of a failed `find`. At `None`, it attaches a new leaf in $\mathcal{O}(1)$ pointer updates. Total time: $\mathcal{O}(H)$.

Trace: `insert(7, "G")`



Key Observation

Unlike dynamic arrays, inserting into a BST does not require shifting existing nodes in memory.

Handling Duplicate Keys in `tree.py`

The Implementation in `tree.py`

In `tree.py`, line 44 writes:

```
1 if key < self.key:
2   ...
3 else:
4   # Routes key >= self.key to right:
5   if self.right is None:
6     self.right = TreeNode(key, value)
7   else:
8     self.right.insert(key, value)
```

If `key == self.key`, it creates a duplicate node in the right subtree.

Effect of Duplicate Keys

Because `find` stops at the first matching key:

- Re-inserting an existing key shadows the new value at a leaf below.
- `lookup` will never reach the updated value.

The Map ADT Correction

In a standard key-value map, matching keys update the value in-place:

```
1 elif key == self.key:
2   self.value = value
```

Preserves uniqueness and updates in $\mathcal{O}(H)$ time.

Handling Duplicate Keys in `tree.py`

The Implementation in `tree.py`

In `tree.py`, line 44 writes:

```
1 if key < self.key:
2   ...
3 else:
4   # Routes key >= self.key to right:
5   if self.right is None:
6     self.right = TreeNode(key, value)
7   else:
8     self.right.insert(key, value)
```

If `key == self.key`, it creates a duplicate node in the right subtree.

Effect of Duplicate Keys

Because `find` stops at the first matching key:

- Re-inserting an existing key shadows the new value at a leaf below.
- `lookup` will never reach the updated value.

The Map ADT Correction

In a standard key-value map, matching keys update the value in-place:

```
1 elif key == self.key:
2   self.value = value
```

Preserves uniqueness and updates in $\mathcal{O}(\mathrm{H})$ time.

Tree Traversals & Python Generators

Depth-First Tree Traversals

Three Classic Depth-First Orders

- **Pre-Order ($\mathbf{N} \to \mathbf{L} \to \mathbf{R}$):** Visit node before subtrees. Used for cloning or serialising tree structure.
- **In-Order ($\mathbf{L} \to \mathbf{N} \to \mathbf{R}$):** Visit left subtree, then node, then right subtree.
- **Post-Order ($\mathbf{L} \to \mathbf{R} \to \mathbf{N}$):** Visit subtrees before node. Used for bottom-up computation (e.g. subtree size).

In-Order Traversal Theorem

An in-order traversal of a BST with distinct keys visits keys in **strictly increasing sorted order**.

Sorted Order via Projection

Visual Intuition

Projecting every node vertically onto a horizontal line reproduces the sorted sequence directly.

Depth-First Tree Traversals

Three Classic Depth-First Orders

- **Pre-Order ($\mathbf{N} \to \mathbf{L} \to \mathbf{R}$):** Visit node before subtrees. Used for cloning or serialising tree structure.
- **In-Order ($\mathbf{L} \to \mathbf{N} \to \mathbf{R}$):** Visit left subtree, then node, then right subtree.
- **Post-Order ($\mathbf{L} \to \mathbf{R} \to \mathbf{N}$):** Visit subtrees before node. Used for bottom-up computation (e.g. subtree size).

In-Order Traversal Theorem

An in-order traversal of a BST with distinct keys visits keys in **strictly increasing sorted order**.

Sorted Order via Projection

Visual Intuition

Projecting every node vertically onto a horizontal line reproduces the sorted sequence directly.

Depth-First Tree Traversals

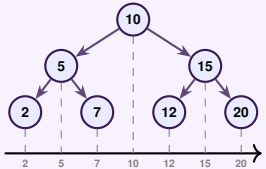
Three Classic Depth-First Orders

- **Pre-Order ($\mathbf{N} \to \mathbf{L} \to \mathbf{R}$):** Visit node before subtrees. Used for cloning or serialising tree structure.
- **In-Order ($\mathbf{L} \to \mathbf{N} \to \mathbf{R}$):** Visit left subtree, then node, then right subtree.
- **Post-Order ($\mathbf{L} \to \mathbf{R} \to \mathbf{N}$):** Visit subtrees before node. Used for bottom-up computation (e.g. subtree size).

In-Order Traversal Theorem

An in-order traversal of a BST with distinct keys visits keys in **strictly increasing sorted order**.

Sorted Order via Projection



Visual Intuition

Projecting every node vertically onto a horizontal line reproduces the sorted sequence directly.

Depth-First Tree Traversals

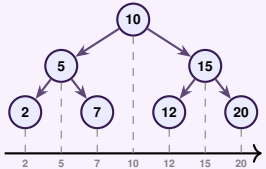
Three Classic Depth-First Orders

- **Pre-Order ($\mathbf{N} \to \mathbf{L} \to \mathbf{R}$):** Visit node before subtrees. Used for cloning or serialising tree structure.
- **In-Order ($\mathbf{L} \to \mathbf{N} \to \mathbf{R}$):** Visit left subtree, then node, then right subtree.
- **Post-Order ($\mathbf{L} \to \mathbf{R} \to \mathbf{N}$):** Visit subtrees before node. Used for bottom-up computation (e.g. subtree size).

In-Order Traversal Theorem

An in-order traversal of a BST with distinct keys visits keys in **strictly increasing sorted order**.

Sorted Order via Projection



Visual Intuition

Projecting every node vertically onto a horizontal line reproduces the sorted sequence directly.

Limitations of Returning a List

Implementation: Returning a List

```
1 def walk_list(self):
2   result = []
3   if self.left is not None:
4     result.extend(self.left.walk_list())
5   result.append((self.key, self.value))
6   if self.right is not None:
7     result.extend(self.right.walk_list())
8   return result
```

Three Drawbacks

1. **Memory Overhead ($\Theta(N)$):** Allocates a list holding all N elements simultaneously.
2. **Initial Latency:** The caller cannot inspect element 1 until all N nodes have been collected.
3. **Early Exit Cost:** If searching for the first matching item, building the remainder of the list is redundant.

The Alternative: Lazy Evaluation

Yield individual items on demand without allocating an intermediate container.

Limitations of Returning a List

Implementation: Returning a List

```
def walk_list(self):
  result = []
  if self.left is not None:
    result.extend(self.left.walk_list())
  result.append((self.key, self.value))
  if self.right is not None:
    result.extend(self.right.walk_list())
  return result
```

Three Drawbacks

1. **Memory Overhead ($\Theta(N)$):** Allocates a list holding all N elements simultaneously.
2. **Initial Latency:** The caller cannot inspect element 1 until all N nodes have been collected.
3. **Early Exit Cost:** If searching for the first matching item, building the remainder of the list is redundant.

The Alternative: Lazy Evaluation

Yield individual items on demand without allocating an intermediate container.

Python Generators and `yield` (`yield-example-1.py`)

```python
def return_example():
  return [1, 2, 3]  # Function terminates

def yield_example():
  yield 1  # Suspends execution
  yield 2  # Resumes here on next()
  yield 3

gen = yield_example()
print(next(gen))  # 1
print(next(gen))  # 2
```

`return` vs `yield`

- **`return`:** Deallocates the activation frame and returns control permanently.
- **`yield`:** Suspends the generator frame on the heap, preserves local variables, and produces a value.

The Iterator Protocol

Calling a generator function returns a **generator iterator**. A `for` loop invokes `next()` repeatedly until `StopIteration` is raised.

Python Generators and `yield` (`yield-example-1.py`)

```python
def return_example():
  return [1, 2, 3]  # Function terminates

def yield_example():
  yield 1  # Suspends execution
  yield 2  # Resumes here on next()
  yield 3

gen = yield_example()
print(next(gen))  # 1
print(next(gen))  # 2
```

`return` vs `yield`

- `return`**:** Deallocates the activation frame and returns control permanently.
- `yield`**:** Suspends the generator frame on the heap, preserves local variables, and produces a value.

The Iterator Protocol

Calling a generator function returns a **generator iterator**. A `for` loop invokes `next()` repeatedly until `StopIteration` is raised.

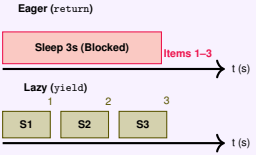
Lazy vs Eager Evaluation (yield-example-2.py)

```python
import time

def return_example():
  result = []
  for i in range(1, 4):
    time.sleep(1) # simulate work
    result.append(i)
  return result

def yield_example():
  for i in range(1, 4):
    time.sleep(1) # simulate work
    yield i
```

Execution Timeline Comparison



Interleaved Execution

Time-to-first-item: 1 s with `yield` vs 3 s with `return`. Execution is single-threaded and interleaved (cooperative suspension and resumption).

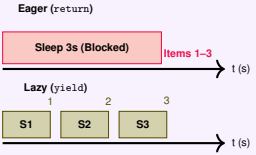
Lazy vs Eager Evaluation (`yield-example-2.py`)

```python
import time

def return_example():
  result = []
  for i in range(1, 4):
    time.sleep(1) # simulate work
    result.append(i)
  return result

def yield_example():
  for i in range(1, 4):
    time.sleep(1) # simulate work
    yield i
```

Execution Timeline Comparison



Interleaved Execution

Time-to-first-item: 1 s with `yield` vs 3 s with `return`. Execution is single-threaded and interleaved (cooperative suspension and resumption).

Recursive Generator Delegation: `yield from`

Manual Iteration Delegation

```
1 # Iterating over a sub-generator:
2 for item in subgenerator():
3   yield item
```

Direct Delegation: PEP 380

```
1 # Direct delegation:
2 yield from subgenerator()
```

How `yield from` Operates

`yield from <iterable>` delegates iteration directly to a sub-generator:

- Values produced by the sub-generator are passed directly to the outer caller.
- Sub-generator termination is handled automatically without manual loop boilerplate.

Recursive Traversals

Enables writing recursive tree traversals directly in clean Python.



Recursive Generator Delegation: `yield from`

Manual Iteration Delegation

```
1 # Iterating over a sub-generator:
2 for item in subgenerator():
3   yield item
```

Direct Delegation: PEP 380

```
1 # Direct delegation:
2 yield from subgenerator()
```

How `yield from` Operates

`yield from <iterable>` delegates iteration directly to a sub-generator:

- Values produced by the sub-generator are passed directly to the outer caller.
- Sub-generator termination is handled automatically without manual loop boilerplate.

Recursive Traversals

Enables writing recursive tree traversals directly in clean Python.



In-Order BST Generator (`TreeNode.walk`)

```python
class TreeNode:
  def walk(self):
    if self.left is not None:
      yield from self.left.walk()
    yield (self.key, self.value)
    if self.right is not None:
      yield from self.right.walk()

# Consuming the generator:
for key, val in tree.root.walk():
  print(key, val)
```

Time and Space Complexity

- **Total Visits:** $\Theta(N)$ node visits in total.
- **First Item Latency:** $\mathcal{O}(H)$ to walk down to the minimum key.
- **Auxiliary Memory:** $\Theta(H)$ suspended generator frames on the heap along the active path ($\Theta(\log N)$ balanced, $\Theta(N)$ degenerate).

Heap Memory Savings

In a balanced BST of 10^6 nodes, `walk()` maintains ≈ 20 suspended frames, compared to an eager list holding 10^6 tuples.

In-Order BST Generator (`TreeNode.walk`)

```python
class TreeNode:
  def walk(self):
    if self.left is not None:
      yield from self.left.walk()
    yield (self.key, self.value)
    if self.right is not None:
      yield from self.right.walk()

# Consuming the generator:
for key, val in tree.root.walk():
  print(key, val)
```

Time and Space Complexity

- **Total Visits:** $\Theta(N)$ node visits in total.
- **First Item Latency:** $\mathcal{O}(H)$ to walk down to the minimum key.
- **Auxiliary Memory:** $\Theta(H)$ suspended generator frames on the heap along the active path ($\Theta(\log N)$ balanced, $\Theta(N)$ degenerate).

Heap Memory Savings

In a balanced BST of 10^6 nodes, `walk()` maintains ≈ 20 suspended frames, compared to an eager list holding 10^6 tuples.

In-Order BST Generator (`TreeNode.walk`)

```python
class TreeNode:
  def walk(self):
    if self.left is not None:
      yield from self.left.walk()
    yield (self.key, self.value)
    if self.right is not None:
      yield from self.right.walk()

# Consuming the generator:
for key, val in tree.root.walk():
  print(key, val)
```

Time and Space Complexity

- **Total Visits:** $\Theta(N)$ node visits in total.
- **First Item Latency:** $\mathcal{O}(H)$ to walk down to the minimum key.
- **Auxiliary Memory:** $\boldsymbol{\Theta(H)}$ suspended generator frames on the heap along the active path ($\Theta(\log N)$ balanced, $\Theta(N)$ degenerate).

Heap Memory Savings

In a balanced BST of 10^6 nodes, `walk()` maintains ≈ 20 suspended frames, compared to an eager list holding 10^6 tuples.

In-Order BST Generator (`TreeNode.walk`)

```python
class TreeNode:
  def walk(self):
    if self.left is not None:
      yield from self.left.walk()
    yield (self.key, self.value)
    if self.right is not None:
      yield from self.right.walk()

# Consuming the generator:
for key, val in tree.root.walk():
  print(key, val)
```

Time and Space Complexity

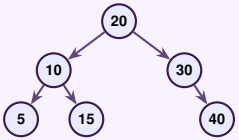
- **Total Visits:** $\Theta(N)$ node visits in total.
- **First Item Latency:** $\mathcal{O}(H)$ to walk down to the minimum key.
- **Auxiliary Memory:** $\boldsymbol{\Theta(H)}$ suspended generator frames on the heap along the active path ($\Theta(\log N)$ balanced, $\Theta(N)$ degenerate).

Heap Memory Savings

In a balanced BST of 10^6 nodes, `walk()` maintains ≈ 20 suspended frames, compared to an eager list holding 10^6 tuples.

Checkpoint 2: Generator Execution & Traversal Orders

Consider This 6-Node BST



Question 1: Lazy Invocation

When running `g = tree.root.walk()`, how many nodes have been visited before calling `next(g)`?

Question 2: Traversal Outputs

Predict the exact output sequence for each order:

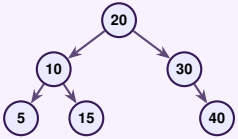
1. **Pre-Order ($\mathrm{N} \to \mathrm{L} \to \mathrm{R}$):**
   $20, 10, 5, 15, 30, 40$
2. **In-Order ($\mathrm{L} \to \mathrm{N} \to \mathrm{R}$):**
   $5, 10, 15, 20, 30, 40$ *(Sorted sequence)*
3. **Post-Order ($\mathrm{L} \to \mathrm{R} \to \mathrm{N}$):**
   $5, 15, 10, 40, 30, 20$

Answer to Question 1

Zero nodes. Calling a generator function returns an iterator without executing the function body until `next()` is called.

Checkpoint 2: Generator Execution & Traversal Orders

Consider This 6-Node BST



Question 1: Lazy Invocation

When running `g = tree.root.walk()`, how many nodes have been visited before calling `next(g)`?

Question 2: Traversal Outputs

Predict the exact output sequence for each order:

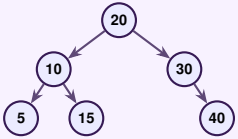
1. **Pre-Order ($\mathbf{N} \to \mathbf{L} \to \mathbf{R}$):**
   $20, 10, 5, 15, 30, 40$
2. **In-Order ($\mathbf{L} \to \mathbf{N} \to \mathbf{R}$):**
   $5, 10, 15, 20, 30, 40$ *(Sorted sequence)*
3. **Post-Order ($\mathbf{L} \to \mathbf{R} \to \mathbf{N}$):**
   $5, 15, 10, 40, 30, 20$

Answer to Question 1

Zero nodes. Calling a generator function returns an iterator without executing the function body until `next()` is called.

Checkpoint 2: Generator Execution & Traversal Orders

Consider This 6-Node BST



Question 1: Lazy Invocation

When running `g = tree.root.walk()`, how many nodes have been visited before calling `next(g)`?

Question 2: Traversal Outputs

Predict the exact output sequence for each order:

1. **Pre-Order ($\mathbf{N} \to \mathbf{L} \to \mathbf{R}$):**
   $20, 10, 5, 15, 30, 40$
2. **In-Order ($\mathbf{L} \to \mathbf{N} \to \mathbf{R}$):**
   $5, 10, 15, 20, 30, 40$ *(Sorted sequence)*
3. **Post-Order ($\mathbf{L} \to \mathbf{R} \to \mathbf{N}$):**
   $5, 15, 10, 40, 30, 20$

Answer to Question 1

Zero nodes. Calling a generator function returns an iterator without executing the function body until `next()` is called.

Advanced BST: Deletion & Hibbard Swapping

The Challenge of BST Deletion

Why Deletion is Hard

- `insert` always adds a new node as a **leaf** at the bottom of the tree.
- `delete` may remove a node from the middle of the tree (or the root itself).
- Remaining nodes must be rewired to preserve the global BST invariant strictly.

Classification by Number of Children

To delete a target node D:

- **Case 1 (0 Children — Leaf):** Discard D.
- **Case 2 (1 Child):** Link D's parent directly to D's single child.
- **Case 3 (2 Children):** Neither child can replace D directly without detaching the other. Requires an in-order replacement.

The Challenge of BST Deletion

Why Deletion is Hard

- `insert` always adds a new node as a **leaf** at the bottom of the tree.
- `delete` may remove a node from the middle of the tree (or the root itself).
- Remaining nodes must be rewired to preserve the global BST invariant strictly.

Classification by Number of Children

To delete a target node D:

- **Case 1 (0 Children — Leaf):** Discard D.
- **Case 2 (1 Child):** Link D's parent directly to D's single child.
- **Case 3 (2 Children):** Neither child can replace D directly without detaching the other. Requires an in-order replacement.

The Challenge of BST Deletion

Why Deletion is Hard

- `insert` always adds a new node as a **leaf** at the bottom of the tree.
- `delete` may remove a node from the middle of the tree (or the root itself).
- Remaining nodes must be rewired to preserve the global BST invariant strictly.

Classification by Number of Children

To delete a target node D:

- **Case 1 (0 Children — Leaf):** Discard D.
- **Case 2 (1 Child):** Link D's parent directly to D's single child.
- **Case 3 (2 Children):** Neither child can replace D directly without detaching the other. Requires an in-order replacement.

The Challenge of BST Deletion

Why Deletion is Hard

- `insert` always adds a new node as a **leaf** at the bottom of the tree.
- `delete` may remove a node from the middle of the tree (or the root itself).
- Remaining nodes must be rewired to preserve the global BST invariant strictly.

Classification by Number of Children

To delete a target node D:

- **Case 1 (0 Children — Leaf):** Discard D.
- **Case 2 (1 Child):** Link D's parent directly to D's single child.
- **Case 3 (2 Children):** Neither child can replace D directly without detaching the other. Requires an in-order replacement.

The Challenge of BST Deletion

Why Deletion is Hard

- `insert` always adds a new node as a **leaf** at the bottom of the tree.
- `delete` may remove a node from the middle of the tree (or the root itself).
- Remaining nodes must be rewired to preserve the global BST invariant strictly.

Classification by Number of Children

To delete a target node D:

- **Case 1 (0 Children — Leaf):** Discard D.
- **Case 2 (1 Child):** Link D's parent directly to D's single child.
- **Case 3 (2 Children):** Neither child can replace D directly without detaching the other. Requires an in-order replacement.

The Challenge of BST Deletion

Why Deletion is Hard

- `insert` always adds a new node as a **leaf** at the bottom of the tree.
- `delete` may remove a node from the middle of the tree (or the root itself).
- Remaining nodes must be rewired to preserve the global BST invariant strictly.

Classification by Number of Children

To delete a target node D:

- **Case 1 (0 Children — Leaf):** Discard D.
- **Case 2 (1 Child):** Link D's parent directly to D's single child.
- **Case 3 (2 Children):** Neither child can replace D directly without detaching the other. Requires an in-order replacement.

The Recursive Pointer-Update Idiom

```
1  class TreeNode:
2    def delete(self, key):
3      if self.key == key:
4        # Handle Cases 1, 2, or 3
5        ...
6      elif key < self.key:
7        if self.left:
8          self.left = self.left.delete(key)
9      else:
10       if self.right:
11         self.right = self.right.delete(key)
12     return self
13
14 class BinaryTree:
15   def delete(self, key):
16     if self.root:
17       self.root = self.root.delete(key)
```

Subtree Root Assignment

Notice the recursive update:

```
self.left = self.left.delete(key)
```

- `delete()` returns the node that should now occupy that position.
- When a child unlinks itself, the parent updates its child pointer directly.
- Eliminates the need for parent pointers in `TreeNode`.

The Recursive Pointer-Update Idiom

```
1  class TreeNode:
2    def delete(self, key):
3      if self.key == key:
4        # Handle Cases 1, 2, or 3
5        ...
6      elif key < self.key:
7        if self.left:
8          self.left = self.left.delete(key)
9      else:
10       if self.right:
11         self.right = self.right.delete(key)
12     return self
13
14 class BinaryTree:
15   def delete(self, key):
16     if self.root:
17       self.root = self.root.delete(key)
```

Subtree Root Assignment

Notice the recursive update:

```
self.left = self.left.delete(key)
```

- `delete()` returns the node that should now occupy that position.
- When a child unlinks itself, the parent updates its child pointer directly.
- Eliminates the need for parent pointers in `TreeNode`.



The Recursive Pointer-Update Idiom

```
1  class TreeNode:
2    def delete(self, key):
3      if self.key == key:
4        # Handle Cases 1, 2, or 3
5        ...
6      elif key < self.key:
7        if self.left:
8          self.left = self.left.delete(key)
9      else:
10       if self.right:
11         self.right = self.right.delete(key)
12     return self
13
14 class BinaryTree:
15   def delete(self, key):
16     if self.root:
17       self.root = self.root.delete(key)
```

Subtree Root Assignment

Notice the recursive update:

```
self.left = self.left.delete(key)
```

- `delete()` returns the node that should now occupy that position.
- When a child unlinks itself, the parent updates its child pointer directly.
- Eliminates the need for parent pointers in `TreeNode`.



Deletion Case 1 (Leaf) and Case 2 (Single Child)

```python
# Inside TreeNode.delete (self.key == key):

# Case 1: Leaf (0 children)
if self.left is None and self.right is None:
  return None

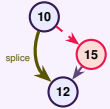
# Case 2a: Only right child exists
elif self.left is None:
  return self.right

# Case 2b: Only left child exists
elif self.right is None:
  return self.left
```

Case 1: Delete Leaf 2



Case 2: Delete Node 15 (Single Child 12)



Invariant Preservation in Case 2

All nodes in D's single child subtree already satisfy the invariant relative to D's parent.

Deletion Case 1 (Leaf) and Case 2 (Single Child)

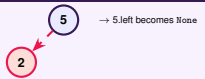
```python
# Inside TreeNode.delete (self.key == key):

# Case 1: Leaf (0 children)
if self.left is None and self.right is None:
  return None

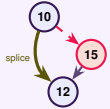
# Case 2a: Only right child exists
elif self.left is None:
  return self.right

# Case 2b: Only left child exists
elif self.right is None:
  return self.left
```

Case 1: Delete Leaf 2



Case 2: Delete Node 15 (Single Child 12)



Invariant Preservation in Case 2

All nodes in D's single child subtree already satisfy the invariant relative to D's parent.

Deletion Case 3: The Two-Child Dilemma

The Replacement Requirement

Target node D has subtrees $T_L \neq \emptyset$ and $T_R \neq \emptyset$. A valid replacement R^* must satisfy:

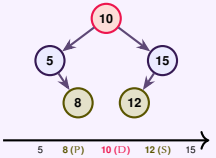
$$\forall x \in T_L \backslash \{R^*\},\ \forall y \in T_R \backslash \{R^*\}: \quad x.\text{key} < R^*.\text{key} < y.\text{key}$$

The Unique Candidates

Only two keys in the tree satisfy this property:

1. **In-Order Predecessor:** $P = \max(T_L)$.
2. **In-Order Successor:** $S = \min(T_R)$.

Adjacent Keys in Sorted Order



Immediate Neighbours

8 and 12 are adjacent to 10 in sorted order.
Promoting either preserves the invariant strictly.

Deletion Case 3: The Two-Child Dilemma

The Replacement Requirement

Target node D has subtrees $T_L \neq \emptyset$ and $T_R \neq \emptyset$. A valid replacement R^* must satisfy:

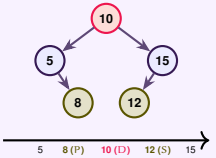
$$\forall x \in T_L \backslash \{R^*\},\ \forall y \in T_R \backslash \{R^*\}: \quad x.\text{key} < R^*.\text{key} < y.\text{key}$$

The Unique Candidates

Only two keys in the tree satisfy this property:

1. **In-Order Predecessor:** $P = \max(T_L)$.
2. **In-Order Successor:** $S = \min(T_R)$.

Adjacent Keys in Sorted Order



Immediate Neighbours

8 and 12 are adjacent to 10 in sorted order.
Promoting either preserves the invariant strictly.

Deletion Case 3: The Two-Child Dilemma

The Replacement Requirement

Target node D has subtrees $T_L \neq \emptyset$ and $T_R \neq \emptyset$. A valid replacement R^* must satisfy:

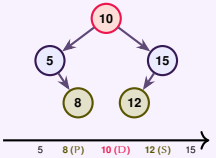
$$\forall x \in T_L \backslash \{R^*\},\ \forall y \in T_R \backslash \{R^*\}: \quad x.\text{key} < R^*.\text{key} < y.\text{key}$$

The Unique Candidates

Only two keys in the tree satisfy this property:

1. **In-Order Predecessor:** $P = \max(T_L)$.
2. **In-Order Successor:** $S = \min(T_R)$.

Adjacent Keys in Sorted Order



Immediate Neighbours

8 and 12 are adjacent to 10 in sorted order.
Promoting either preserves the invariant strictly.

Finding the Predecessor & Predecessor Child Property

Locating the Predecessor (`maxkey`)

```
1 class TreeNode:
2   def maxkey(self):
3     if self.right is None:
4       return self.key
5     return self.right.maxkey()
```

To find the maximum in a subtree, follow `right` pointers until `node.right is None`.

Lemma: Predecessor Has No Right Child

The in-order predecessor $P = \max(D.\text{left})$ **cannot have a right child** ($P.\text{right} = \texttt{None}$).

Proof by Contradiction

If P had a right child, that child would have key $> P.\text{key}$, contradicting that P is the maximum key in D's left subtree. □

Termination Guarantee

Because P has at most one child (it can have a **left child**), deleting P from $D.\text{left}$ is **guaranteed** to reduce to Case 1 or Case 2.

Finding the Predecessor & Predecessor Child Property

Locating the Predecessor (`maxkey`)

```
1 class TreeNode:
2   def maxkey(self):
3     if self.right is None:
4       return self.key
5     return self.right.maxkey()
```

To find the maximum in a subtree, follow `right` pointers until `node.right is None`.

Lemma: Predecessor Has No Right Child

The in-order predecessor $\mathrm{P} = \max(\mathrm{D}.\mathrm{left})$ **cannot have a right child** ($\mathrm{P}.\mathrm{right} = \mathtt{None}$).

Proof by Contradiction

If P had a right child, that child would have key $> \mathrm{P}.\mathrm{key}$, contradicting that P is the maximum key in D's left subtree. □

Termination Guarantee

Because P has at most one child (it can have a **left child**), deleting P from $\mathrm{D}.\mathrm{left}$ is **guaranteed** to reduce to Case 1 or Case 2.

Finding the Predecessor & Predecessor Child Property

Locating the Predecessor (`maxkey`)

```
1 class TreeNode:
2   def maxkey(self):
3     if self.right is None:
4       return self.key
5     return self.right.maxkey()
```

To find the maximum in a subtree, follow `right` pointers until `node.right is None`.

Lemma: Predecessor Has No Right Child

The in-order predecessor $\mathrm{P} = \max(\mathrm{D}.\mathrm{left})$ **cannot have a right child** ($\mathrm{P}.\mathrm{right} = \mathtt{None}$).

Proof by Contradiction

If P had a right child, that child would have key $> \mathrm{P}.\mathrm{key}$, contradicting that P is the maximum key in D's left subtree. $\square$

Termination Guarantee

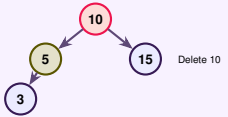
Because P has at most one child (it can have a **left child**), deleting P from $\mathrm{D}.\mathrm{left}$ is **guaranteed** to reduce to Case 1 or Case 2.

Strategy A: Hibbard Value Swapping

`tree-fixed-question-response.py`

```python
# Case 3 in tree-fixed-question-response.py:
else:
  # 1. Locate predecessor key and node
  maxkey = self.left.maxkey()
  maxnode = self.left.find(maxkey)
  # 2. Overwrite target node's data in-place
  self.key = maxnode.key
  self.value = maxnode.value
  # 3. Recursively delete predecessor from left
  self.left = self.left.delete(maxkey)
  return self # Target node stays in place
```

Step-by-Step on 4-Node Tree



Result After Value Swap

Thomas N. Hibbard (1962)

Avoids physical pointer splicing. We overwrite the payload and delete the predecessor at the bottom.

Strategy A: Hibbard Value Swapping

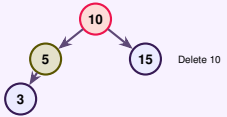
`tree-fixed-question-response.py`

```python
# Case 3 in tree-fixed-question-response.py:
else:
  # 1. Locate predecessor key and node
  maxkey = self.left.maxkey()
  maxnode = self.left.find(maxkey)
  # 2. Overwrite target node's data in-place
  self.key = maxnode.key
  self.value = maxnode.value
  # 3. Recursively delete predecessor from left
  self.left = self.left.delete(maxkey)
  return self # Target node stays in place
```

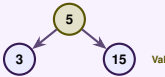
Thomas N. Hibbard (1962)

Avoids physical pointer splicing. We overwrite the payload and delete the predecessor at the bottom.

Step-by-Step on 4-Node Tree



Result After Value Swap



Strategy B: Pointer Splicing & The Bug in `tree.py`

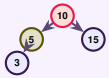
Lines 91–97 of `tree.py`

```
1 maxkey = self.left.maxkey()
2 maxnode = self.left.find(maxkey)
3 self.left = self.left.delete(maxkey)
4 assert(maxnode.left is None and
5        maxnode.right is None) # BUG
6 maxnode.left = self.left
7 maxnode.right = self.right
8 return maxnode
```

The Erroneous Assertion

The code asserts that predecessor `maxnode` has no children. But P can have a **left child**.

Checkpoint 3: Crash Trace



What Crashes on `delete(10)`?

1. Predecessor is 5, where `maxnode.left` points to 3.
2. `delete(5)` unlinks 5, but the misplaced assert checks `maxnode.left` before line 6 rewires it.
3. Line 94 raises `AssertionError`.

Removing the assert fixes splicing; value swapping avoids node splicing entirely.

Strategy B: Pointer Splicing & The Bug in `tree.py`

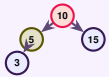
Lines 91–97 of `tree.py`

```
1 maxkey = self.left.maxkey()
2 maxnode = self.left.find(maxkey)
3 self.left = self.left.delete(maxkey)
4 assert(maxnode.left is None and
5        maxnode.right is None) # BUG
6 maxnode.left = self.left
7 maxnode.right = self.right
8 return maxnode
```

The Erroneous Assertion

The code asserts that predecessor `maxnode` has no children. But P can have a **left child**.

Checkpoint 3: Crash Trace



What Crashes on `delete(10)`?

1. Predecessor is 5, where `maxnode.left` points to 3.
2. `delete(5)` unlinks 5, but the misplaced assert checks `maxnode.left` before line 6 rewires it.
3. Line 94 raises `AssertionError`.

Removing the assert fixes splicing; value swapping avoids node splicing entirely.

Engineering Refinements & Hibbard Degradation

Single-Pass Predecessor Lookup

Notice that both scripts call:

`maxkey()` then `find(maxkey)`

This traverses the right spine of the left subtree **twice**.

- Writing a helper `maxnode()` that returns the node directly achieves the lookup in a **single pass**.
- Still $\mathcal{O}(\mathrm{H})$, but halves the constant factor.

Hibbard's $\Theta(\sqrt{\mathrm{N}})$ Degradation (Eppinger, 1983)

Always selecting the predecessor from the **left** subtree causes an asymmetry:

- Over $\Theta(\mathrm{N}^2)$ random insertions and deletions, the tree gradually skews right.
- Average height degrades from $\mathcal{O}(\log \mathrm{N})$ to $\boldsymbol{\Theta}(\sqrt{\mathbf{N}})$.

Mitigation

Alternate between predecessor and successor, or use self-balancing trees.

Engineering Refinements & Hibbard Degradation

Single-Pass Predecessor Lookup

Notice that both scripts call:

`maxkey()` then `find(maxkey)`

This traverses the right spine of the left subtree **twice**.

- Writing a helper `maxnode()` that returns the node directly achieves the lookup in a **single pass**.
- Still $\mathcal{O}(\mathrm{H})$, but halves the constant factor.

Hibbard's $\Theta(\sqrt{\mathrm{N}})$ Degradation (Eppinger, 1983)

Always selecting the predecessor from the **left** subtree causes an asymmetry:

- Over $\Theta(\mathrm{N}^2)$ random insertions and deletions, the tree gradually skews right.
- Average height degrades from $\mathcal{O}(\log \mathrm{N})$ to $\boldsymbol{\Theta}(\sqrt{\mathbf{N}})$.

Mitigation

Alternate between predecessor and successor, or use self-balancing trees.

Summary of BST Deletion Cases

Case	Structural Condition	Action Taken	Complexity
Case 1: Leaf	0 children (`left` is `None`, `right` is `None`).	Return `None` to parent. Parent drops pointer.	$\mathcal{O}(\mathrm{H})$
Case 2: 1 Child	Exactly 1 child (`left` or `right` is `None`).	Return non-empty child. Parent bypasses deleted node.	$\mathcal{O}(\mathrm{H})$
Case 3: 2 Children	2 children (`left` and `right` both present).	Copy predecessor (k, v) into node; recursively delete predecessor.	$\mathcal{O}(\mathrm{H})$

Preserving the BST Invariant

All three cases preserve the global BST ordering property. Because Case 3 reduces to Case 1 or 2, at most two downward paths are traversed, bounding total time by $\mathcal{O}(\mathrm{H})$.

Complexity & The Balance Trap

Asymptotic Complexity of BST Operations

Summary of Operations

Operation	Balanced Tree	Degenerate Tree
`lookup(k)`	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$
`insert(k, v)`	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$
`delete(k)`	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$
`minkey()` / `maxkey()`	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$
`walk()` (all keys)	$\Theta(N)$	$\Theta(N)$

Single-operation bounds are worst-case over arbitrary inputs; root operations take $\Theta(1)$.

Height Governs Traversal Work

Every search or update operation follows a single root-to-leaf path:

$$T(N) = \mathcal{O}(H)$$

The Power of Balanced Trees

For $N = 10^6$ items:

- Linear search ($\mathcal{O}(N)$): Up to 10^6 comparisons.
- Balanced BST ($\mathcal{O}(\log_2 N)$): At most $\lfloor \log_2(10^6) \rfloor + 1 = \mathbf{20}$ comparisons.

Dependence on Insertion Order

Standard BSTs do not guarantee balance. Their shape depends entirely on the insertion order of keys.

Asymptotic Complexity of BST Operations

Summary of Operations

Operation	Balanced Tree	Degenerate Tree
`lookup(k)`	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$
`insert(k, v)`	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$
`delete(k)`	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$
`minkey()` / `maxkey()`	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$
`walk()` (all keys)	$\Theta(N)$	$\Theta(N)$

Single-operation bounds are worst-case over arbitrary inputs; root operations take $\Theta(1)$.

Height Governs Traversal Work

Every search or update operation follows a single root-to-leaf path:

$$T(N) = \mathcal{O}(H)$$

The Power of Balanced Trees

For $N = 10^6$ items:

- Linear search ($\mathcal{O}(N)$): Up to 10^6 comparisons.
- Balanced BST ($\mathcal{O}(\log_2 N)$): At most $\lfloor \log_2(10^6) \rfloor + 1 = \mathbf{20}$ comparisons.

Dependence on Insertion Order

Standard BSTs do not guarantee balance. Their shape depends entirely on the insertion order of keys.

Worst Case: Degenerate Trees

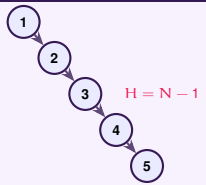
What Determines Tree Shape?

If keys arrive in random order, expected height is $\mathcal{O}(\log N)$. But what if keys arrive **already sorted**?

```
1 tree.insert(1, "A")
2 tree.insert(2, "B")
3 tree.insert(3, "C")
4 tree.insert(4, "D")
```

Every key is larger than the previous one, so every insertion branches `right`.

The Linked List Degeneration



Degradation to $\mathcal{O}(N)$

The tree degenerates into a singly linked list. `lookup`, `insert`, and `delete` all degrade to $\Theta(N)$ linear scans.

Worst Case: Degenerate Trees

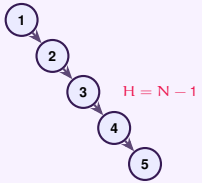
What Determines Tree Shape?

If keys arrive in random order, expected height is $\mathcal{O}(\log N)$. But what if keys arrive **already sorted**?

```
1 tree.insert(1, "A")
2 tree.insert(2, "B")
3 tree.insert(3, "C")
4 tree.insert(4, "D")
```

Every key is larger than the previous one, so every insertion branches `right`.

The Linked List Degeneration



Degradation to $\mathcal{O}(N)$

The tree degenerates into a singly linked list. `lookup`, `insert`, and `delete` all degrade to $\Theta(N)$ linear scans.

Checkpoint 4: Inserting Sorted Keys

Real-World Scenario

A web service logs customer transactions into a standard BST, using the order's **auto-incrementing ID** or **timestamp** as the key:

$$k_1 < k_2 < k_3 < \cdots < k_N$$

Diagnostic Questions

1. What is the height H after inserting N transactions?
2. What is the **total cumulative time** to insert all N transactions?

Mathematical Derivation

- Height: $H = N - 1$.
- The i-th insertion traverses $i - 1$ edges:

$$T(N) = \sum_{i=1}^{N} \Theta(i) = \Theta(\frac{N(N-1)}{2}) = \Theta(N^2)$$

Practical Consequence

Chronological data is common in engineering. An unbalanced BST turns an expected $\mathcal{O}(N \log N)$ workload into a quadratic $\Theta(N^2)$ runtime.



Checkpoint 4: Inserting Sorted Keys

Real-World Scenario

A web service logs customer transactions into a standard BST, using the order's **auto-incrementing ID** or **timestamp** as the key:

$$k_1 < k_2 < k_3 < \cdots < k_N$$

Diagnostic Questions

1. What is the height H after inserting N transactions?
2. What is the **total cumulative time** to insert all N transactions?

Mathematical Derivation

- Height: $H = N - 1$.
- The i-th insertion traverses $i - 1$ edges:

$$T(N) = \sum_{i=1}^{N} \Theta(i) = \Theta(\frac{N(N-1)}{2}) = \boldsymbol{\Theta(N^2)}$$

Practical Consequence

Chronological data is common in engineering. An unbalanced BST turns an expected $\mathcal{O}(N \log N)$ workload into a quadratic $\Theta(N^2)$ runtime.

Guaranteeing $\mathcal{O}(\log N)$: Self-Balancing Trees

Self-Balancing Invariants

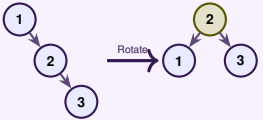
To prevent degeneracy, self-balancing BSTs enforce structural invariants:

- **AVL Trees (1962):** For every node, heights of left and right subtrees differ by at most 1 ($|\text{balance}| \leqslant 1$).
- **Red-Black Trees (1978):** Colour rules guarantee no root-to-leaf path is more than twice as long as any other.

Tree Rotations

When an insertion or deletion unbalances the tree, local $\mathcal{O}(1)$ pointer updates (**rotations**) restore balance without violating the BST invariant.

Left Rotation Mechanics



Guaranteed Worst-Case Bounds

Self-balancing trees guarantee $H = \mathcal{O}(\log N)$ in the worst case for **all** operations.

Guaranteeing $\mathcal{O}(\log N)$: Self-Balancing Trees

Self-Balancing Invariants

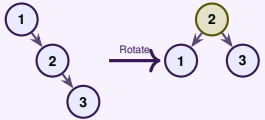
To prevent degeneracy, self-balancing BSTs enforce structural invariants:

- **AVL Trees (1962):** For every node, heights of left and right subtrees differ by at most 1 ($|\text{balance}| \leqslant 1$).
- **Red-Black Trees (1978):** Colour rules guarantee no root-to-leaf path is more than twice as long as any other.

Tree Rotations

When an insertion or deletion unbalances the tree, local $\mathcal{O}(1)$ pointer updates (**rotations**) restore balance without violating the BST invariant.

Left Rotation Mechanics



Guaranteed Worst-Case Bounds

Self-balancing trees guarantee $H = \mathcal{O}(\log N)$ in the worst case for **all** operations.

Beyond Key Comparisons

Comparison Search Lower Bound

In Week 3, we showed that comparison-based sorting requires $\Omega(N \log N)$ comparisons.

- Similarly, any comparison-based search among N items requires a decision tree of height at least:

$$H \geqslant \lceil \log_2(N+1) \rceil - 1 = \Omega(\log N)$$

- No comparison tree can achieve $\mathcal{O}(1)$ lookup in the worst case.

Direct Memory Addressing

Instead of **comparing** keys against each other, compute the memory index directly from the key.

Hash Tables

A hash function converts a key directly into an array index:

$$k \xrightarrow{\text{hash}} \text{index} \implies \mathcal{O}(1) \text{ expected lookup}$$

Beyond Key Comparisons

Comparison Search Lower Bound

In Week 3, we showed that comparison-based sorting requires $\Omega(N \log N)$ comparisons.

- Similarly, any comparison-based search among N items requires a decision tree of height at least:

$$H \geqslant \lceil \log_2(N+1) \rceil - 1 = \boldsymbol{\Omega}(\log \mathbf{N})$$

- No comparison tree can achieve $\mathcal{O}(1)$ lookup in the worst case.

Direct Memory Addressing

Instead of **comparing** keys against each other, compute the memory index directly from the key.

Hash Tables

A hash function converts a key directly into an array index:

$$k \xrightarrow{\text{hash}} \text{index} \implies \mathcal{O}(\mathbf{1}) \text{ expected lookup}$$

Hash Tables and Collision Resolution

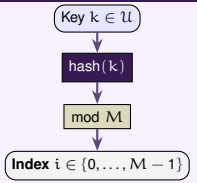
From Direct Addressing to Hash Functions

Direct Addressing

If keys are small integers $k \in \{0, 1, \ldots, M-1\}$, store each item in an array `A[k]`.

- Lookup, insertion, and deletion are strictly $\mathcal{O}(1)$.
- **Limitation:** The universe of possible keys $\mathcal{U}$ (e.g. 64-bit integers, strings) is enormous ($|\mathcal{U}| \geqslant 2^{64}$), making full direct arrays impractical.

The Two-Step Hashing Pipeline



Concrete Example

"Steve" $\xrightarrow{\text{hash}}$ -452187321 $\xrightarrow{\text{mod } 4}$ Index 3.

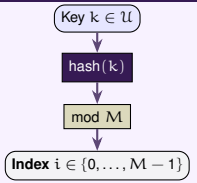
From Direct Addressing to Hash Functions

Direct Addressing

If keys are small integers $k \in \{0, 1, \ldots, M-1\}$, store each item in an array `A[k]`.

- Lookup, insertion, and deletion are strictly $\mathcal{O}(1)$.
- **Limitation:** The universe of possible keys $\mathcal{U}$ (e.g. 64-bit integers, strings) is enormous ($|\mathcal{U}| \geqslant 2^{64}$), making full direct arrays impractical.

The Two-Step Hashing Pipeline



Concrete Example

`"Steve"` $\xrightarrow{\text{hash}}$ `-452187321` $\xrightarrow{\text{mod } 4}$ Index 3.

The Pigeonhole Principle & Hash Collisions

The Inevitability of Collisions

Because the universe of keys $|\mathcal{U}|$ is much larger than table size M:

$$|\mathcal{U}| > M$$

by the **Pigeonhole Principle**, there must exist at least two distinct keys $k_1 \neq k_2$ that hash to the exact same bucket:

$$\text{hash}(k_1) \bmod M = \text{hash}(k_2) \bmod M$$

Collision Resolution Strategies

1. **Separate Chaining:** Each bucket stores a list of colliding elements (`hashtable.py`).
2. **Open Addressing:** All elements live in the table array itself. Collisions probe alternative empty slots using a deterministic probe sequence (CPython `dict`).

Design Rule

A hash table is defined by its hash function and collision resolution policy.

The Pigeonhole Principle & Hash Collisions

The Inevitability of Collisions

Because the universe of keys $|\mathcal{U}|$ is much larger than table size M:

$$|\mathcal{U}| > M$$

by the **Pigeonhole Principle**, there must exist at least two distinct keys $k_1 \neq k_2$ that hash to the exact same bucket:

$$\text{hash}(k_1) \bmod M = \text{hash}(k_2) \bmod M$$

Collision Resolution Strategies

1. **Separate Chaining:** Each bucket stores a list of colliding elements (`hashtable.py`).
2. **Open Addressing:** All elements live in the table array itself. Collisions probe alternative empty slots using a deterministic probe sequence (CPython `dict`).

Design Rule

A hash table is defined by its hash function and collision resolution policy.

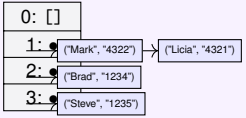
Separate Chaining Architecture (`hashtable.py`)

```
1  class HashTable:
2    def __init__(self, element_list):
3      # Assumes non-empty; else max(1, ...)
4      n = len(element_list) * 2 - 1
5      self.bucket_count = n
6      self.buckets = [[] for _ in range(n)]
7      for key, value in element_list:
8        self.insert(key, value)
9
10   def get_index(self, key):
11     return hash(key) % self.bucket_count
```

Implementation Note

`hashtable.py` stores bucket chains in Python dynamic arrays (`list`), whereas classical separate chaining uses linked lists.

Memory Model ($M = 4$)



Collision in Bucket 1

"Mark" and "Licia" both hash to bucket 1 (hash mod $4 = 1$) and form a chain.

Separate Chaining: Insert and Lookup

```python
class HashTable:
  def insert(self, key, value):
    idx = self.get_index(key)
    # Check for existing key:
    for i, (k, v) in enumerate(self.buckets[idx])
     :
      if k == key:
        self.buckets[idx][i] = (key, value)
        return
    self.buckets[idx].append((key, value))

  def lookup(self, search_key):
    idx = self.get_index(search_key)
    for key, value in self.buckets[idx]:
      if key == search_key:
        return value
    return None
```

Storing Both Key and Value

Multiple keys map to the same bucket.

- Bucket 1 contains `("Mark", ...)` and `("Licia", ...)`.
- During `lookup("Licia")`, each entry's key is checked to find the matching value.

Worst-Case Collision Time

If all N keys hash to the same bucket, the chain has length N, degrading search to $\Theta(N)$.

Separate Chaining: Insert and Lookup

```python
class HashTable:
  def insert(self, key, value):
    idx = self.get_index(key)
    # Check for existing key:
    for i, (k, v) in enumerate(self.buckets[idx])
     :
      if k == key:
        self.buckets[idx][i] = (key, value)
        return
    self.buckets[idx].append((key, value))

  def lookup(self, search_key):
    idx = self.get_index(search_key)
    for key, value in self.buckets[idx]:
      if key == search_key:
        return value
    return None
```

Storing Both Key and Value

Multiple keys map to the same bucket.

- Bucket 1 contains `("Mark", ...)` and `("Licia", ...)`.
- During `lookup("Licia")`, each entry's key is checked to find the matching value.

Worst-Case Collision Time

If all N keys hash to the same bucket, the chain has length N, degrading search to $\Theta(N)$.

Load Factor (α) & Expected $\mathcal{O}(1)$ Complexity

Definition of Load Factor (α)

For a table with N stored items and M buckets, the **load factor** is:

$$\alpha = \frac{N}{M}$$

α represents the **average number of items per bucket**.

Simple Uniform Hashing Assumption (SUHA)

Assuming $\mathcal{O}(1)$-time hash evaluation, each key is equally likely to hash into any of the M slots:

$$\Pr(h(k) \bmod M = j) = \frac{1}{M} \quad \forall j \in \{0, 1, \ldots, M-1\}$$

Expected Search Time Under SUHA

In separate chaining under SUHA:

$$T_{\text{expected}} = \Theta(1 + \alpha)$$

- 1: Compute hash and index bucket array ($\mathcal{O}(1)$).
- α: Scan expected chain length ($\frac{N}{M}$).

Bounding the Load Factor

If table resizing ensures $\alpha \leqslant \alpha_{\text{max}} = \mathcal{O}(1)$ (e.g. $\alpha_{\text{max}} = 0.75$), then $\Theta(1 + \alpha) = \boldsymbol{\Theta(1)}$ expected time.

Load Factor (α) & Expected $\mathcal{O}(1)$ Complexity

Definition of Load Factor (α)

For a table with N stored items and M buckets, the **load factor** is:

$$\alpha = \frac{N}{M}$$

α represents the **average number of items per bucket**.

Simple Uniform Hashing Assumption (SUHA)

Assuming $\mathcal{O}(1)$-time hash evaluation, each key is equally likely to hash into any of the M slots:

$$\Pr(h(k) \bmod M = j) = \frac{1}{M} \quad \forall j \in \{0, 1, \ldots, M-1\}$$

Expected Search Time Under SUHA

In separate chaining under SUHA:

$$T_{\text{expected}} = \Theta(1 + \alpha)$$

- 1: Compute hash and index bucket array ($\mathcal{O}(1)$).
- α: Scan expected chain length ($\frac{N}{M}$).

Bounding the Load Factor

If table resizing ensures $\alpha \leqslant \alpha_{\max} = \mathcal{O}(1)$ (e.g. $\alpha_{\max} = 0.75$), then $\Theta(1 + \alpha) = \boldsymbol{\Theta(1)}$ expected time.

Dynamic Resizing & Amortised Analysis

When to Resize (Rehashing)

As elements are inserted, $\alpha = N/M$ grows. When $\alpha > \alpha_{max}$:

1. Allocate a new table of size $M' = 2M$.
2. Iterate through all N existing elements.
3. Recompute $\text{hash}(k)$ mod M' for every key and re-insert into new buckets.

Why Rehashing is Required

Because $\text{hash}(k) \bmod M \neq \text{hash}(k) \bmod (2M)$, bucket elements must be redistributed.

Amortised Cost Derivation

A rehash costs $\Theta(N + M') = \Theta(N)$ work.

- Capacity doubles geometrically ($M \to 2M \to 4M$).
- An $\mathcal{O}(N)$ rehash occurs only after $N/2$ insertions:

$$T_{\text{amortised}} = \frac{(N/2) \cdot \mathcal{O}(1) + \Theta(N)}{N/2} = \mathcal{O}(\mathbf{1})$$

Conclusion

Insertion is $\mathcal{O}(1)$ amortised expected under SUHA; lookup is $\mathcal{O}(1)$ expected.

Dynamic Resizing & Amortised Analysis

When to Resize (Rehashing)

As elements are inserted, $\alpha = N/M$ grows. When $\alpha > \alpha_{max}$:

1. Allocate a new table of size $M' = 2M$.
2. Iterate through all N existing elements.
3. Recompute hash(k) mod M' for every key and re-insert into new buckets.

Why Rehashing is Required

Because hash(k) mod $M \neq$ hash(k) mod $(2M)$, bucket elements must be redistributed.

Amortised Cost Derivation

A rehash costs $\Theta(N + M') = \Theta(N)$ work.

- Capacity doubles geometrically ($M \to 2M \to 4M$).
- An $\mathcal{O}(N)$ rehash occurs only after $N/2$ insertions:

$$T_{\text{amortised}} = \frac{(N/2) \cdot \mathcal{O}(1) + \Theta(N)}{N/2} = \mathcal{O}(\mathbf{1})$$

Conclusion

Insertion is $\mathcal{O}(1)$ amortised expected under SUHA; lookup is $\mathcal{O}(1)$ expected.

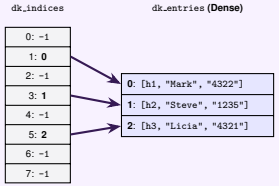
Under the Hood: CPython 3.6+ Compact Dictionaries

Open Addressing in CPython

CPython uses open addressing with a compact two-array layout:

- `dk_indices`: Sparse array of signed integers ($M = 8, 16, \ldots$).
- `dk_entries`: Dense array of $[hash, key, value]$ rows, appended in **insertion order**.
- **Probing on Collision:** If an index slot is occupied, CPython evaluates a perturbed probe sequence to find the next available index.

Two-Array Architecture



Two Major Wins

30%–55% less RAM and **guaranteed insertion-order iteration** at zero extra cost.

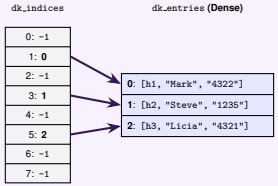
Under the Hood: CPython 3.6+ Compact Dictionaries

Open Addressing in CPython

CPython uses open addressing with a compact two-array layout:

- `dk_indices`: Sparse array of signed integers ($M = 8, 16, \ldots$).
- `dk_entries`: Dense array of $[hash, key, value]$ rows, appended in **insertion order**.
- **Probing on Collision:** If an index slot is occupied, CPython evaluates a perturbed probe sequence to find the next available index.

Two-Array Architecture



Two Major Wins

30%–55% less RAM and **guaranteed insertion-order iteration** at zero extra cost.

The Hashability Contract in Python (`dict.py`)

```python
# Valid hashable keys:
d = {10: "int", "UCL": "string"}
t = (1, 2, "tuple_ok")
d[t] = "valid"

# Invalid mutable keys:
lst = [1, 2]
d[lst] = "fails"
# TypeError: unhashable type: 'list'

# Mixed tuple with mutable contents:
d[(1, [2, 3])] = "fails too"
# TypeError: unhashable type: 'list'
```

The Formal Hashability Contract

An object k is hashable if and only if:

1. **Lifetime Invariance:** `hash(k)` remains constant over its lifetime.
2. **Equality Coherence:** $a = b \implies \text{hash}(a) = \text{hash}(b)$.
3. **Classes & Containers:** User classes hash by identity by default. Built-in containers require immutable elements.

Crucial Note

$\text{hash}(a) = \text{hash}(b) \not\Rightarrow a = b$. That is simply a hash collision.

The Hashability Contract in Python (`dict.py`)

```python
# Valid hashable keys:
d = {10: "int", "UCL": "string"}
t = (1, 2, "tuple_ok")
d[t] = "valid"

# Invalid mutable keys:
lst = [1, 2]
d[lst] = "fails"
# TypeError: unhashable type: 'list'

# Mixed tuple with mutable contents:
d[(1, [2, 3])] = "fails too"
# TypeError: unhashable type: 'list'
```

The Formal Hashability Contract

An object k is hashable if and only if:

1. **Lifetime Invariance:** `hash(k)` remains constant over its lifetime.
2. **Equality Coherence:** $a = b \implies \text{hash}(a) = \text{hash}(b)$.
3. **Classes & Containers:** User classes hash by identity by default. Built-in containers require immutable elements.

Crucial Note

$\text{hash}(a) = \text{hash}(b) \not\Rightarrow a = b$. That is simply a hash collision.

Checkpoint 5: Mutating a Dictionary Key

Thought Experiment

Suppose Python allowed mutable lists as dictionary keys in an $M = 8$ table:

```
k = [10, 20]
d[k] = "secret"
# hash([10, 20]) % 8 == 2 -> Bucket 2

# Mutate key in-place:
k.append(30) # k is now [10, 20, 30]
```

The Diagnostic Question

What happens when evaluating `d[k]` or `d[[10, 20]]`?

Unreachable Entry

Lookup Failure

- `d[k]`: Computes `hash([10,20,30]) % 8 == 6`. Bucket 6 is empty $\Longrightarrow$ `KeyError`.
- `d[[10,20]]`: Probes Bucket 2, but key has changed to `[10,20,30]` $\Longrightarrow$ `KeyError`.

The entry is permanently unreachable.

Checkpoint 5: Mutating a Dictionary Key

Thought Experiment

Suppose Python allowed mutable lists as dictionary keys in an $M = 8$ table:

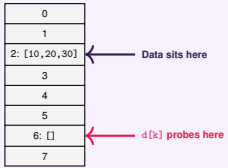
```
k = [10, 20]
d[k] = "secret"
# hash([10, 20]) % 8 == 2 -> Bucket 2

# Mutate key in-place:
k.append(30) # k is now [10, 20, 30]
```

The Diagnostic Question

What happens when evaluating `d[k]` or `d[[10, 20]]`?

Unreachable Entry



Lookup Failure

- `d[k]`: Computes `hash([10,20,30]) % 8 == 6`. Bucket 6 is empty $\Longrightarrow$ `KeyError`.
- `d[[10,20]]`: Probes Bucket 2, but key has changed to `[10,20,30]` $\Longrightarrow$ `KeyError`.

The entry is permanently unreachable.

Synthesis, Comparison & Outlook

Data Structure Complexity Summary (Weeks 2–4)

Data Structure	Search	Insert	Delete	Sorted Iteration	Ordered By
Unsorted Array (`list`)	$\mathcal{O}(N)$	$\mathcal{O}(1)^{A}$	$\mathcal{O}(N)$	$\mathcal{O}(N \log N)$	Insertion order
Sorted Array (`list`)	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$	$\Theta(N)$	Key order
Linked Deque (`deque`)	$\mathcal{O}(N)$	$\mathcal{O}(1)$ at ends	$\mathcal{O}(1)$ at ends	$\mathcal{O}(N \log N)$	Insertion order
Balanced BST (AVL / RB)	$\mathcal{O}(\log \mathbf{N})$	$\mathcal{O}(\log \mathbf{N})$	$\mathcal{O}(\log \mathbf{N})$	$\boldsymbol{\Theta}(\mathbf{N})$	Key order
Degenerate BST (Worst)	$\mathcal{O}(N)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$	$\Theta(N)$	Key order
Hash Table (`dict`)	$\mathcal{O}(\mathbf{1})$ exp.	$\mathcal{O}(\mathbf{1})^{\mathbf{A}}$ exp.	$\mathcal{O}(\mathbf{1})$ exp.	$\mathcal{O}(N \log N)$	Insertion order

When to Prefer BSTs

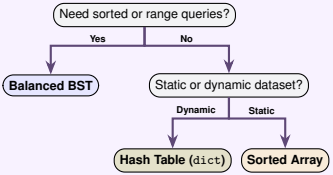
Need range queries ($\texttt{min} \leqslant \texttt{k} \leqslant \texttt{max}$), predecessor/successor, or streaming sorted data without extra sorting passes.

When to Prefer Hash Tables

Need point lookups (`d[key]`) as fast as possible ($\mathcal{O}(1)$ average time), and do not require sorted order.

Choosing a Data Structure

Decision Flowchart

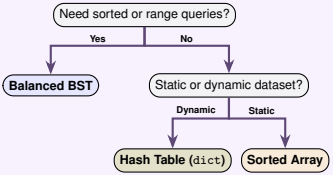


Common Applications

- **Databases (PostgreSQL, SQLite):** Use B+Trees for index range scans; Hash indexes for point queries.
- **Python Runtime:** Variable namespaces and `globals()` are implemented as compact dictionaries.
- **Symbol Tables:** Scoped dictionaries mapping variable names to types.

Choosing a Data Structure

Decision Flowchart



Common Applications

- **Databases (PostgreSQL, SQLite):** Use B+Trees for index range scans; Hash indexes for point queries.
- **Python Runtime:** Variable namespaces and `globals()` are implemented as compact dictionaries.
- **Symbol Tables:** Scoped dictionaries mapping variable names to types.

Key Takeaways & Looking Ahead

Core Lecture Takeaways

1. **BST Invariant:** Strictly partitions search spaces to achieve $\mathcal{O}(\mathrm{H})$ operations.
2. **Hibbard Deletion:** Reduces 2-child deletion to Case 1 or 2 by swapping predecessor data.
3. **Python Generators:** `yield` streams items lazily in $\Theta(\mathrm{H})$ memory, eliminating eager allocation.
4. **Hash Tables:** Achieve $\mathcal{O}(1)$ expected lookup under uniform hashing by bounding load factor $\alpha \leqslant \alpha_{\max}$.

Connecting to Assignment 4

In Assignment 4 (Tetris Autoplayer), you will use Python **generators (`yield`)** to stream candidate rotation and translation moves lazily toward target landing positions.

Preview: Week 5

Next week, we move beyond data storage trees into **Search and Game Trees** (mazes, depth-first search, and the Minimax algorithm).

Key Takeaways & Looking Ahead

Core Lecture Takeaways

1. **BST Invariant:** Strictly partitions search spaces to achieve $\mathcal{O}(H)$ operations.
2. **Hibbard Deletion:** Reduces 2-child deletion to Case 1 or 2 by swapping predecessor data.
3. **Python Generators:** `yield` streams items lazily in $\Theta(H)$ memory, eliminating eager allocation.
4. **Hash Tables:** Achieve $\mathcal{O}(1)$ expected lookup under uniform hashing by bounding load factor $\alpha \leqslant \alpha_{\max}$.

Connecting to Assignment 4

In Assignment 4 (Tetris Autoplayer), you will use Python **generators (`yield`)** to stream candidate rotation and translation moves lazily toward target landing positions.

Preview: Week 5

Next week, we move beyond data storage trees into **Search and Game Trees** (mazes, depth-first search, and the Minimax algorithm).